

Aprendizaje Composicional: Guía Introductoria

Un mapa de lectura para la Investigación en Pregrado

Preparado a partir de los materiales del curso MAT2320

Este documento es una guía de entrada al tema de *Compositional Learning* (aprendizaje composicional), pensada para que puedas comenzar tu investigación con un mapa claro de: (i) qué es el problema, (ii) cómo se conecta con Machine Learning y Reinforcement Learning, (iii) qué dice cada uno de los cuatro documentos que ya tienes, y (iv) qué leer después para profundizar.

1. ¿Qué es el aprendizaje composicional?

La intuición es simple: un sistema **compone** cuando resuelve problemas nuevos *combinando piezas que ya conoce*, en vez de necesitar ver el problema completo durante el entrenamiento. Un niño que aprendió “salta” y “dos veces” por separado entiende “salta dos veces” sin haber visto nunca ese ejemplo. Una red neuronal entrenada de la forma estándar, en cambio, frecuentemente falla en exactamente este tipo de combinación nueva, aunque haya memorizado perfectamente los ejemplos de entrenamiento.

Formalmente (Documento C, §4.3), si $\varphi : \Sigma_{in}^* \rightarrow \Sigma_{out}^*$ es la función que el sistema debería aprender, decimos que φ es composicional si es un **homomorfismo de monoides**:

$$\varphi(a \circ b) = \varphi(a) \cdot \varphi(b).$$

Es decir, el significado de un todo se obtiene combinando el significado de las partes mediante una regla fija. Un modelo *generaliza composicionalmente* si aplica esa misma regla a combinaciones (a, b) que nunca vio juntas, siempre que ya sepa resolver a y b por separado.

¿Por qué importa? Porque es la propiedad que distingue *aprender un mecanismo* de *memorizar patrones*. Un sistema que solo memoriza necesita ver (en el límite) todas las combinaciones posibles; un sistema composicional necesita ver solo las piezas, y el número de piezas crece mucho más lento que el número de combinaciones. Esta es la razón profunda por la que la composicionalidad se conecta con **eficiencia de muestra**, **generalización fuera de distribución (OOD)** y, en el fondo, con la **complejidad de Kolmogorov**: una descripción composicional de los datos es, casi por definición, una descripción más corta.

2. Relación con Machine Learning

En ML supervisado clásico se optimiza el error empírico sobre un conjunto de entrenamiento, asumiendo que entrenamiento y prueba vienen de la misma distribución $\mathcal{D}$. La composicionalidad pregunta algo distinto: ¿qué pasa cuando el conjunto de prueba contiene *combinaciones nuevas* de elementos conocidos? Ahí es donde aparecen los famosos fracasos de modelos que en apariencia “funcionan”: bajan de $\sim 100\%$ a menos de 2% de exactitud en *splits* composicionales (SCAN, ver §5). Esto es un caso particular de **aprendizaje por atajos** (*shortcut learning*): el modelo encontró una correlación espuria que funciona en la distribución de entrenamiento pero no captura el mecanismo real.

Dos ideas de ML están directamente al servicio de la composicionalidad:

- **Representaciones desacopladas (*disentangled*)**: si cada dimensión latente captura un factor independiente del mundo (color, forma, posición), recombinar factores nuevos es trivial. El Documento C cita el teorema de imposibilidad de Locatello et al. (2019): *sin sesgo inductivo, el desacoplamiento no es identificable a partir de datos puros*. Esto motiva imponer estructura (discreción, dispersión, tipos) en vez de esperar que emerja sola.
- **Representaciones discretas/dispersas**: el Documento B muestra experimentalmente que codificar el espacio latente como vectores *multi-one-hot* (binarios y dispersos) –en vez de vectores continuos densos– permite modelar más del mundo con menos capacidad, justamente porque facilita la recombinación de piezas discretas.

3. Relación con Reinforcement Learning

En RL el problema se agrava: el agente no solo debe generalizar sobre datos estáticos, sino sobre una secuencia de **decisiones** en un mundo que puede ser enorme, parcialmente observado y cambiante. Aquí la composicionalidad aparece de tres formas relacionadas, que son justamente las que cubren tus documentos:

1. **Composicionalidad en la representación del estado.** Un *world model* (modelo del mundo) que aprende representaciones discretas y reutilizables de las observaciones puede, en principio, recombinarlas para imaginar situaciones nuevas. Esto es el corazón de DreamerV3 (Documento A) y del análisis crítico del Documento B.
2. **Composicionalidad en el comportamiento.** Políticas o habilidades aprendidas para una tarea pueden, en un agente verdaderamente composicional, reutilizarse como bloques para tareas nuevas (esto es la motivación detrás del aprendizaje continuo y jerárquico, que el Documento C formaliza en §4.11 mediante el *olvido catastrófico* y EWC).
3. **Composicionalidad explícita en programas.** En vez de que la composicionalidad emerja (o no) de una red neuronal, se puede construir un sistema que *literalmente* mantiene una biblioteca de funciones reutilizables y resuelve tareas combinándolas. Este es DreamCoder (Documento D), que aunque nació como síntesis de programas, es exactamente el caso “puro” de aprendizaje composicional, y se ha aplicado a entornos que son, en esencia, problemas de planeación/decisión (Logo graphics, ARC-AGI).

El RL es, por tanto, el terreno donde la pregunta “¿composicionalidad implícita (redes neuronales) o explícita (programas/símbolos)?” se vuelve más concreta y más urgente, porque un mundo como Minecraft o un entorno continuo (*continual RL*) literalmente no se puede memorizar entero: el agente *debe* aprender piezas reusables para sobrevivir a la novedad.

4. Mapa de los cuatro documentos

A continuación, qué es cada documento, qué aprenderás en él y por qué está ahí.

4.1. Documento C — “Aprendizaje Composicional y Generalización OOD” (Notas Clase 16, Petrache)

Rol en tu lectura: este es el capítulo de *teoría y vocabulario*. Conviene leerlo primero y volver a él como referencia constante.

Qué aprenderás:

- El marco de la **complejidad de Kolmogorov** $K(x \mid y)$ y el **principio MDL** (*Minimum Description Length*): la idea de que “inteligente” puede formalizarse como “capaz de describir/resolver tareas con programas cortos”.
- Por qué esta medida ideal es **no computable** (Teorema 2.4), y por qué eso obliga a usar *síntomas observables* en su lugar.
- Un catálogo de **benchmarks** estándar: SCAN, COGS, gSCAN (composicionalidad en lenguaje); CLEVR-CoGenT, RAVEN, Relational Games (razonamiento visual/relacional); ImageNet-C/R, WILDS, DomainBed (robustez OOD); ARC-AGI, LARC (razonamiento abstracto). Si tu proyecto necesita evaluar un sistema, probablemente usará alguno de estos.
- Once **síntomas formalizados** de inteligencia algorítmica, cada uno con su definición y teorema asociado: atajos espurios (IRM), generalización OOD (cota de Ben-David), generalización composicional (homomorfismo, las 5 propiedades de Hupkes et al.), extrapolación, eficiencia de muestra (PAC/VC), razonamiento causal (jerarquía de Pearl), desacoplamiento (imposibilidad de Locatello), razonamiento relacional/analógico, robustez adversarial, meta-aprendizaje (MAML), aprendizaje continuo (olvido catastrófico, EWC).

Lo esencial en una frase: la inteligencia genuina puede pensarse como *compresión eficiente y reutilizable de la experiencia*, y este documento te da las herramientas matemáticas para medir cuándo un sistema realmente la tiene.

4.2. Documento D — “DreamCoder and Wake-Sleep Library Learning” (Notas Clase 18, Petrache)

Rol en tu lectura: el caso *paradigmático* de aprendizaje composicional *explícito*. Si el Documento C te da el lenguaje, este te da el algoritmo de referencia.

Qué aprenderás:

- Cómo DreamCoder (Ellis et al., 2021) trata la **biblioteca de primitivas** L como una variable latente que se aprende junto con los programas que resuelven cada tarea, minimizando una versión concreta del objetivo MDL del Documento C.
- El algoritmo de **wake-sleep** en tres fases: *Wake* (buscar programas guiados por una red neuronal + la probabilidad bajo L), *Sleep/Abstraction* (extraer subrutinas reutilizables si compensan su costo, ej. descubrir `map` a partir de dos programas parecidos), *Sleep/Dreaming* (entrenar la red de reconocimiento con tareas reales y con “fantasías” generadas por el propio modelo).
- La conexión formal de wake-sleep con la **máquina de Helmholtz** y los **autoencoders variacionales** (Tabla 1 del documento): mismo esquema de inferencia variacional, pero con variable latente *discreta* (un programa) en lugar de un vector continuo, y con un prior (la biblioteca) que *también se aprende*, en vez de estar fijo.

- Aplicaciones a **ARC-AGI** (PeARL, sistemas híbridos LLM+DreamCoder) y la tensión “rápido vs. inteligente”: la red neuronal debe ahorrar más búsqueda de la que cuesta computacionalmente, igual que Stockfish dominó el ajedrez antes de AlphaZero.
- Una tabla de cierre que conecta *cada* concepto del Documento C (Kolmogorov, composicionalidad, OOD, causalidad, meta-aprendizaje, aprendizaje continuo) con su instanciación concreta dentro de DreamCoder. Esa tabla es, en cierto sentido, el resumen más útil de todo el set de lecturas.

Lo esencial en una frase: DreamCoder muestra cómo construir, de forma totalmente explícita y verificable, un sistema que crece su propio vocabulario de soluciones reutilizables — la versión “de libro de texto” de lo que las redes neuronales hacen (si lo hacen) de forma implícita y opaca.

4.3. Documento A — “Mastering Diverse Domains through World Models” (DreamerV3; Hafner, Pasukonis, Ba, Lillicrap, 2023)

Rol en tu lectura: el contraste *neuronal/implícito* frente a DreamCoder. Es un paper de investigación (no notas de clase), así que es más técnico y orientado a resultados experimentales de RL.

Qué aprenderás:

- Qué es un **modelo del mundo** (*world model*) en RL moderno: una red que (i) codifica observaciones en una representación latente *categorica/discreta* z_t , (ii) predice cómo evoluciona esa representación con un estado recurrente h_t (arquitectura RSSM), y (iii) permite que el agente “imagine” trayectorias futuras sin tocar el entorno real.
- Cómo un actor y un crítico aprenden **enteramente dentro de esas imaginaciones**, y por qué esto hace que el algoritmo sea muy eficiente en datos.
- Un conjunto de **trucos de robustez** (transformación **symlog**, pérdida **twohot**, normalización de retornos, balance de KL) que permiten usar *los mismos hiperparámetros* en más de 150 tareas de 8 dominios distintos —algo muy poco común en RL, donde normalmente cada entorno requiere su propio ajuste.
- El resultado insignia: Dreamer es el primer algoritmo en obtener diamantes en Minecraft completamente desde cero, sin datos humanos ni currículo, solo con recompensas dispersas.

Por qué está en tu lista: porque la representación latente discreta de Dreamer es exactamente el objeto que el Documento B analiza y cuestiona: ¿realmente ayuda por ser discreta, o por alguna otra propiedad? Dreamer es el “sujeto de estudio”.

Lo esencial en una frase: es el estado del arte en RL basado en modelos del mundo con representaciones discretas, y el punto de partida experimental de tu segundo documento.

4.4. Documento B — “Harnessing Discrete Representations for Continual RL” (Meyer, White, Machado, 2023/2024)

Rol en tu lectura: probablemente el más cercano a tu proyecto, porque hace exactamente el tipo de pregunta experimental que un trabajo de investigación de pregrado puede reproducir o extender.

Qué aprenderás:

- Una comparación controlada de **cuatro tipos de representación**: autoencoder continuo “vanilla”, autoencoder con activación dispersa (FTA), VQ-VAE (discreta, binaria, dispersa) y entrenamiento *end-to-end*, en dos roles: aprender un modelo del mundo y aprender una política (PPO) en entornos MiniGrid.
- El resultado central: la ventaja de lo discreto **no es la discreción en sí**, sino la estructura *multi-one-hot* (binaria y dispersa). Lo demuestran comparando dos representaciones con *exactamente la misma información* –cuantizada vs. multi-one-hot– y mostrando que solo la segunda generaliza bien (Documento B, §3.2.3).
- Que esta ventaja solo aparece cuando el modelo tiene **capacidad limitada** respecto al tamaño del mundo: con suficiente capacidad, continuo y discreto empatan. Esto conecta directamente con el Documento C: es la versión empírica de “MDL importa cuando los recursos son escasos”.
- En **RL continuo** (el entorno cambia de configuración periódicamente), las representaciones dispersas/discretas se adaptan más rápido tras cada cambio –evidencia de que ayudan a *recombinar* conocimiento viejo en situaciones nuevas, que es justamente la definición operacional de composicionalidad del Documento C.

Lo esencial en una frase: no toda representación discreta es igual de útil; lo que realmente facilita la generalización y la adaptación rápida es la dispersión y el formato binario tipo “multi-one-hot”, y este efecto se vuelve crítico exactamente en el régimen de capacidad limitada que motiva el aprendizaje continual.

5. Cómo se conectan los cuatro documentos

Hay un hilo conductor claro, y vale la pena tenerlo en mente desde el principio:

Documento C	da el lenguaje formal (MDL, Kolmogorov, los once síntomas) para decidir si un sistema generaliza genuinamente.
Documento D	instancia ese lenguaje de la forma más explícita posible: una biblioteca de programas que crece exactamente cuando comprime, vía wake-sleep.
Documento A	ofrece la maquinaria neuronal moderna (RSSM, representaciones categóricas) donde la composicionalidad sería implícita , distribuida en pesos de una red, sin biblioteca explícita.
Documento B	toma esa maquinaria y le hace exactamente la pregunta del Documento C: ¿qué propiedad de la representación (¿discreción? ¿dispersión?) es la que realmente produce eficiencia tipo MDL y generalización composicional? Y responde con un experimento controlado.

En otras palabras: **C** te da la pregunta y las métricas; **D** te muestra la respuesta “de libro” (símbolos explícitos); **A** te muestra la maquinaria neuronal de punta; **B** aplica el rigor experimental de C para diseccionar a A. Tu propio proyecto probablemente se sitúa en algún punto de esta línea: quizás preguntando si se puede combinar la garantía de corrección y la interpretabilidad de D con la escalabilidad de A, usando el tipo de análisis cuidadoso de B y el vocabulario de C.

6. Glosario mínimo

- **MDL (*Minimum Description Length*)**: elegir la hipótesis que minimiza (tamaño de la hipótesis) + (tamaño del error residual). Aproximación computable a la complejidad de Kolmogorov.
- **VQ-VAE**: autoencoder que cuantiza su espacio latente a un conjunto finito de vectores (“code-book”), produciendo representaciones discretas.
- **RSSM (*Recurrent State-Space Model*)**: arquitectura de modelo del mundo que combina un estado recurrente determinista h_t con una representación latente estocástica z_t .
- **PCFG (*Probabilistic Context-Free Grammar*)**: gramática donde cada regla de producción tiene una probabilidad; en DreamCoder, la biblioteca L define una PCFG tipada sobre programas.
- **Wake-sleep**: algoritmo de inferencia variacional con dos fases que se alternan: una ajusta el modelo generativo/los datos (*wake*), otra entrena un modelo de reconocimiento sobre datos reales y sintéticos (*sleep*).
- **OOD (*Out-of-Distribution*)**: generalizar a datos que provienen de una distribución distinta a la de entrenamiento.
- **Aprendizaje continuo (*continual learning*)**: aprender una secuencia de tareas o un entorno que cambia, sin acceso simultáneo a todo el historial, evitando el *olvido catastrófico*.

7. Literatura recomendada

7.1. Para empezar (introdutorio, antes o junto con tus documentos)

- Lake, B. M., Ullman, T. D., Tenenbaum, J. B., & Gershman, S. J. (2017). *Building machines that learn and think like people*. Behavioral and Brain Sciences, 40. — El mejor punto de partida no técnico: por qué la composicionalidad, la causalidad y la construcción de modelos del mundo son centrales para la inteligencia, escrito como debate con comentarios de otros investigadores.
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2. ed.). MIT Press. Caps. 1, 3, 8 (planificación y modelos de aprendizaje) y 17 (opciones / RL jerárquico). Disponible gratis en el sitio del autor. — La base de RL que necesitas para entender los Documentos A y B.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. Cap. 15 (*Representation Learning*). — Para entender autoencoders, representaciones desacopladas y por qué la estructura del espacio latente importa.
- Chollet, F. (2019). *On the Measure of Intelligence*. arXiv:1911.01547. — Ya citado en tus notas; léelo completo, es corto y muy claro, y es la base de ARC-AGI.

7.2. Para profundizar (después de los cuatro documentos)

- Lake, B. M., & Baroni, M. (2018). *Generalization without systematicity*. ICML. — El paper original de SCAN; muy corto y es la cita obligatoria de todo el campo.
- Hupkes, D., Dankers, V., Mul, M., & Bruni, E. (2020). *Compositionality decomposed: How do*

neural networks generalise? JAIR, 67. — La taxonomía de las cinco propiedades de composicionalidad usada en el Documento C; muy citado y muy claro.

- Andreas, J., Rohrbach, M., Darrell, T., & Klein, D. (2016). *Neural Module Networks*. CVPR. — El ejemplo clásico de *composicionalidad explícita dentro de una red neuronal*: cada módulo es una subred reutilizable, y la arquitectura se ensambla dinámicamente según la pregunta. Buen contraste con DreamCoder.
- Battaglia, P. W., et al. (2018). *Relational inductive biases, deep learning, and graph networks*. arXiv:1806.01261. — La referencia estándar sobre cómo incorporar sesgos composicionales/relacionales directamente en la arquitectura (graph networks). Muy relevante si tu proyecto toca representación estructurada de estados en RL.
- Lake, B. M., Salakhutdinov, R., & Tenenbaum, J. B. (2015). *Human-level concept learning through probabilistic program induction*. Science, 350(6266). — El antecedente directo de DreamCoder (Bayesian Program Learning); más corto y más fácil de leer primero.
- Ellis, K., et al. (2021). *DreamCoder: Bootstrapping inductive program synthesis with wake-sleep library learning*. PLDI. — El paper completo detrás del Documento D; vale la pena leerlo entero una vez entendidas las notas.
- Hafner, D., et al. (2018, 2020). *Learning Latent Dynamics for Planning from Pixels* (PlaNet) y *Mastering Atari with Discrete World Models* (DreamerV2). — Las dos generaciones anteriores de Dreamer; ayudan a entender de dónde vienen las decisiones de diseño del Documento A.
- Schölkopf, B., et al. (2021). *Toward Causal Representation Learning*. Proceedings of the IEEE, 109(5). — Conecta composicionalidad, causalidad y disentanglement con más profundidad matemática que las notas de clase.
- Li, M., & Vitányi, P. (2008). *An Introduction to Kolmogorov Complexity and Its Applications* (3. ed.). Springer. Caps. 1–2. — El libro de referencia detrás de toda la Sección 2 del Documento C, si quieres el tratamiento matemático completo.
- Kirkpatrick, J., et al. (2017). *Overcoming catastrophic forgetting in neural networks* (EWC). PNAS. — El paper completo detrás de la subsección de aprendizaje continuo del Documento C, directamente relevante para el Documento B.

8. Ruta de lectura sugerida

1. **Documento C** completo (te da el vocabulario; no te preocupes si algunas demostraciones son densas la primera vez, vuelve a ellas después).
2. Lake et al. (2017), *Building machines...* — para fijar la intuición sin matemática pesada.
3. **Documento D** (DreamCoder) — el caso explícito; ya tendrás el lenguaje de MDL fresco del paso 1.
4. Lake, Salakhutdinov & Tenenbaum (2015) y, si quieres más profundidad, Ellis et al. (2021) completo.
5. **Documento A** (DreamerV3) — el caso neuronal/implícito; concéntrate en entender qué es z_t , h_t y por qué se usan categóricas discretas.

6. **Documento B** — ahora podrás ver inmediatamente cómo aplica el marco de C a la maquinaria de A. Es, en cierto sentido, la lectura “bisagra” de todo el set.
7. Hupkes et al. (2020) y el SCAN original de Lake & Baroni (2018), para profundizar en la formalización de composicionalidad.
8. Según hacia dónde derive tu proyecto: Andreas et al. (2016) si te interesa composicionalidad explícita en arquitecturas neuronales, o Battaglia et al. (2018) si te interesa representación estructurada de estados en RL.